

מה ניתן להחליף בין מודלים, מה לא, ומה צריך כדי שהכל יהיה ניתן להחלפה

מיפוי מדויק — לפי קריאה ישירה בקוד, לא לפי מה שהתיעוד (RFC-01) מתאר
באופן עקרוני — של מה כבר גמיש בין ספקי מודל, מה נעול ל-Claude בכוונה, ומה
נעול רק כי אף אחד עוד לא חיבר אותו.

כל 25 שלבי ה-agent הקיימים במערכת (X, LinkedIn, Reddit, Blog וכו') מוצהרים כ-Claude בלבד – המילה המדויקת `claude-sonnet-4-6` כתובה בקוד המקור של כל אחד מהם. זו לא מגבלה טכנית שנשארה בטעות; ה-RFC המקורי (RFC-01 §5.4) קובע את זה כהחלטת עיצוב. אבל קריאה ישירה במימוש בפועל (`base-agent.ts`) מגלה שהמנגנון עצמו כבר סוכן-אגנוסטי – הוא לא תלוי ב-Claude Agent SDK כפי שה-RFC מתאר, אלא לולאת ReAct עצמאית שקוראת לאותה ממשק `ModelRouter` בלי קשר לספק. המשמעות: המעבר למודל אחר הוא **עבודת חיווט וקוד ממוקדת, לא שינוי ארכיטקטוני**.

המסמך הזה מפריד בין שתי שאלות שונות שקל לערבב: **"איזה צינור"** (transport/auth) קריאות ל-Claude עוברות בו, לעומת **איזו משפחת מודל** (Claude מול Gemini מול GPT) עושה את העבודה בפועל. הראשונה כבר ניתנת להחלפה היום. השנייה – לא, וזה מוסבר למה, ומה בדיוק צריך לבנות כדי שכן.

שתי שכבות שונות של "החלפת מודל"

שכבה	מה זה שולט	מצב היום
המסלול (route) create-model-router-from-env.ts	איך קריאה ל-Claude יוצאת: ישירות ל-Anthropic API (מפתח), או דרך Google Cloud Agent Platform (ADC), ללא מפתח	ניתן להחלפה היום – משתנה סביבה אחד
משפחת המודל (provider) Claude, Gemini, GPT	איזה מודל בפועל מנסח את התוכן – Claude, Gemini, GPT	לא ניתן להחלפה היום עבור עבודת הניסוח בפועל (ראו בהמשך)

כשדיברנו על "חיברתי את הפרויקט ל-Agent Platform" – זה תיאור מדויק של השורה הראשונה בטבלה בלבד. זה לא נגע לשורה השנייה כלל.

מה כבר ניתן להחלפה היום

1. המסלול אל Claude עצמו

`MODEL_PROVIDER=agent-platform` (ברירת מחדל) או `MODEL_PROVIDER=anthropic` – משתנה סביבה יחיד, בלי קוד, בלי הוספת מפתח בצד Agent Platform (ADC בלבד). זה מה שנבנה בשיחה הקודמת.

2. שכבות `commodity` / `portable` – קיימות אבל כמעט לא בשימוש

ל-RFC-01 יש שכבת מדיניות שלישית ורביעית מעבר ל-`pinned`, שמיועדות בדיוק למעבר קליל בין ספקים: סיווג, extraction, embeddings – לא עבודת ניסוח שמגיעה ללקוח. אלה כבר מתחברות לכל endpoint בסגנון OpenAI (`OPENAI_COMPATIBLE_BASE_URL`) – כולל OpenAI האמיתי, וגם Gemini עצמו, שחושף endpoint תואם-OpenAI רשמי (`ai.google.dev/gemini-api/docs/openai`) בלי צורך באדפטר חדש בכלל.

אבל – מתוך 25 שלבי ה-agent הקיימים, רק אחד משתמש בשכבה הזו בפועל: `agents/reputation-agent/src/agent/reputation-extraction-` `claude-haiku-4-5-20251001` – כתוב כקבוע בקוד של אותו `agent.ts`, בשכבת `commodity`. וגם שם, מזהה המודל עצמו –

agent. כלומר: גם כאן, החלפת ספק בפועל דורשת שינוי שורה אחת בקובץ הזה (למזהה מודל של Gemini/GPT), לא רק משתנה סביבה. שאר ה-24 steps כלל לא נוגעים בשכבה הזו.

3. טבלת תמחור כבר "יוזעת" על Gemini ו-GPT

packages/core/src/telemetry/pricing.ts כבר מכילה שורות מחיר עבור `gpt-4o`, `gpt-4o-mini`, `gemini-2.5-flash`, `gemini-2.5-pro` — כך שברגע שספק אחר ייכנס לשימוש, חישוב העלות לא ידרוש עבודה נוספת.

מה לא ניתן להחלפה היום, ולמה

שכבת ה-Claude 100% — pinned, ב-25 מקומות נפרדים בקוד

כל agent אמיתי במערכת (X, LinkedIn, Reddit, Blog, Newsletter, Instagram, SEO/GEO, Intel Report, Branded Shorts, Landing Builder, Reputation, Campaign Orchestrator) מצהיר את שלב הניסוח שלו כך:

```
modelPolicy: { policy: "pinned", model: "claude-sonnet-4-6" }
```

המחרוזת הזו כתובה ישירות בקובץ המקור של כל שלב — לא נטענת ממקום מרכזי אחד. שינוי משפחת מודל לעבודת ניסוח אמיתית מחייב עריכת קוד ב-25 מקומות, לא משתנה סביבה אחד.

תיקון חשוב לעומת מה שנאמר בשיחה הקודמת: §5.4 RFC-01 מתאר את שכבת ה-`pinned` כתלויה ב-Claude Agent SDK — subagents, skills, hooks, permission modes — ומצדיק את הנעילה ל-Claude בזה. אבל קריאה ישירה ב-`packages/core/src/agent/base-agent.ts` מראה שהמימוש בפועל **לא** פותח subagent של Claude Agent SDK בכלל. זו לולאת ReAct עצמאית, כתובה ב-TypeScript טהור, שקוראת ל-`runtime.router.complete()` — בדיוק אותה קריאה ששכבות `portable` / `commodity` משתמשות בה. המשמעות: ברמת המנגנון, אין שום דבר שמונע משכבת ה-`pinned` לקרוא לאדפטר של ספק אחר. הנעילה היא **החלטת חיווט + מדיניות** (הפונקציה שבונה את הראוטר תמיד בונה את `pinned` מ-Claude), לא מגבלת ארכיטקטורה קשיחה. זה חדשות טובות — המרחק לפתרון קצר יותר ממה שה-RFC מרמז.

אין אדפטר טבעי ל-Gemini

קיימים היום שני אדפטרים בלבד המממשים את ממשק `ModelAdapter`: אחד לצורת ה-Messages API של Anthropic (משרת גם את Claude הישיר וגם את Agent Platform — אותה צורת בקשה, לקוח שונה), ואחד לצורת ה-Chat Completions של OpenAI. Gemini (לא דרך שכבת התאימות ל-OpenAI) מדבר צורה שלישית — `response_mime_type` / `response_schema` — ששונה משתייה. אין כרגע קוד שמתרגם אליה.

Prompt caching נשבר בין ספקים

ה-cache של Anthropic הוא per-model, per-endpoint. מעבר ספק לשלב מסוים משמעו איפוס ה-cache עבורו — לא שגיאה, אבל עלות גבוהה יותר עד שנצבר cache חדש. שווה לקחת בחשבון בהחלטת תזמון, לא רק בהחלטת "אפשר/אי אפשר".

מה צריך לעשות כדי שהכל יהיה ניתן להחלפה

סדר מומלץ, מהזול והמהיר ביותר לערך, ועד המקיף ביותר:

1 להוציא את מזהה המודל מכל agent במקום מרכזי אחד

כמו `router/aliases.ts` הקיים כבר עבור ה-table alias של Dynamic Agent Studio — כל ה-25 השלבים יקראו `modelPolicy` מטבלה אחת לפי תפקיד (למשל `DRAFT_MODEL_POLICY`), במקום 25 מחרוזות עצמאיות. זה לא משנה שום התנהגות — זה מה שהופך

את כל שאר הצעדים לזולים.

2

להחליט: להשתמש ב-endpoint התואם OpenAI של Gemini, או לבנות אדפטר Gemini טבעי

התואם- (ai.google.dev/gemini-api/docs/openai) OpenAI (עובד **כבר עכשיו** עם `OpenAICompatibleAdapter` הקיים – משתנה סביבה בלבד, אפס קוד חדש. אדפטר טבעי דורש כתיבת קוד חדש (כמו `agent-platform-adapter.ts` שנבנה עבור Claude) אבל חושף יכולות ש-Gemini תומך בהן רק בצורה הטבעית שלו (context caching מסוג אחר, grounding, וכו'). מומלץ להתחיל בתואם-OpenAI – זול, מהיר, מוכיח את זרימת העבודה – ולעבור לאדפטר טבעי רק אם צריך יכולת ספציפית שחסרה שם.

3

להרכיב את `createModelRouterFromEnv` כך ששכבת ה-pinned תוכל לבחור ספק

היום הפונקציה תמיד בונה את ה-pinned מ-Claude (Agent Platform או ישיר). להוסיף ענף שבו `modelPolicy.policy === "pinned"` יכול להיפתר גם דרך `OpenAICompatibleAdapter` כשמוגדר ספק אחר במפורש – זה שינוי ממוקד באותו קובץ שכבר עודכן עבור Agent Platform.

4

להריץ evals לפני כל מעבר בפועל

§12 RFC-01 כבר מגדיר את המנגנון (golden runs, deterministic assertions, judge model) בדיוק בשביל השאלה "האם המעבר למודל אחר שומר על איכות?" – לא להסתמך על תחושה. סף מומלץ שם: אין רגרסיה באיכות, עלייה בעלות מוגבלת ל-15%.

5

להריץ shadow-run לפני חשיפה ללקוח

להריץ את ה-agent המיועד על אותם קלטים אמיתיים מול שני הספקים במקביל, ולהשוות פלט – לפני שמחליטים שהמודל החדש "עובד", לא רק שהוא מחזיר JSON תקין.

שורה תחתונה

ניתן להחלפה היום – `env var` בלבד

המסלול אל Claude (Agent Platform ↔ ישיר)

כמעט מוכן – `agent 1`, שורת קוד אחת לשינוי

שכבות סיווג/ (`commodity/portable`) `extraction`

דורש עבודה – לא היום

שכבת ניסוח בפועל (pinned) – Claude מול Gemini/GPT

חיווט + 25 מחרוזות מודל, לא ארכיטקטורה

המחסום האמיתי

מבוסס על קריאה ישירה של `packages/core/src/router/agent/base-agent.ts`, `*/packages/core/src/agent/*`, וכל קבצי ה-agent תחת `agents/*/src/agent/*.ts` בענף

`dev` של `agent-engine`, ב-20 באוגוסט 2026.